

SCS 1309

Database Management Systems

Lecture 2 – Structured Query Language (SQL)

MVPT Lakshika,
Lecturer (Probationary),
Department of Information Systems Engineering, UCSC.
Contact: tlv@ucsc.cmb.ac.lk

Outline of this session...

SQL Sub-languages or SQL Command Types

1. Data Definition Language(DDL)
2. Data Manipulation Language (DML)

1. Data Definition Language (DDL)

Data Definition Language (DDL)

- Physical Design (specify the database schema)
- Used to create a database, modify its structure, and destroy it when no longer needed.

Example:

CREATE SCHEMA CREATE DOMAIN CREATE TABLE CREATE VIEW CREATE INDEX	ALTER DOMAIN ALTER TABLE	DROP SCHEMA DROP DOMAIN DROP TABLE DROP VIEW DROP INDEX
---	-----------------------------	---

- **CREATE INDEX** and **DROP INDEX** statements are provided by many DBMSs
(Not covered by the SQL standard)

Creating a Database

- The ISO standard does not specify how databases are created, and each dialect generally has a different approach.
- According to the ISO standard, relations and other database objects exist in an **environment**. Each environment consists of one or more **catalogs**, and each catalog consists of a set of **schemas**.
 - A schema is a named collection of database objects that are in some way related to one another.
 - All the objects in the database are described in one schema or another.
 - The objects in a schema can be tables, views, domains, assertions, collations, translations, and character sets.
 - All the objects in a schema have the same owner and share a number of defaults.

Creating a Database

- The standard leaves the mechanism for creating and destroying catalogs as implementation-defined, but provides mechanisms for creating and destroying schemas.
- The basic syntax for creating a database:

```
CREATE SCHEMA [Name | AUTHORIZATION CreatorIdentifier]
```

e.g., The SQL statement for the schema `SqlTests` by the creator of the schema, `Smith`.

```
CREATE SCHEMA SqlTests AUTHORIZATION Smith;
```

Activity 1: Refer to the *DreamHome* case study provided in your reference document. Create the database named 'DreamHome'.

```
CREATE SCHEMA DreamHome;
```

Dropping a Database

- A schema can be destroyed using the DROP SCHEMA statement.

```
DROP SCHEMA Name [RESTRICT | CASCADE]
```

- If **RESTRICT** is specified, which is the default, if neither qualifier is specified, the schema must be empty, or the operation fails.
- If **CASCADE** is specified, it will drop all objects associated with the schema in the order defined above.
- If any of these drop operations fail, the DROP SCHEMA fails.
- The total effect of a DROP SCHEMA with CASCADE can be very extensive and should be carried out only with extreme caution.
- The CREATE and DROP SCHEMA statements are not yet widely implemented.

Creating a Table

- Having created the database structure, now create the table structures for the base relations to be located in the database.
- The basic syntax:

```
CREATE TABLE TableName  
    {(columnName dataType [NOT NULL] [UNIQUE]  
    [DEFAULT defaultOption] [CHECK (searchCondition)] [, ... ]}  
    [PRIMARY KEY (listOfColumns),  
    {[UNIQUE (listOfColumns)] [, ... ]}  
    {[FOREIGN KEY (listOfForeignKeyColumns)  
    REFERENCES ParentTableName [(listOfCandidateKeyColumns)]  
    [MATCH {PARTIAL | FULL}  
    [ON UPDATE referentialAction]  
    [ON DELETE referentialAction]] [, ... ]}  
    {[CHECK (searchCondition)] [, ... ]}}
```

- This CREATE TABLE statement incorporates facilities for defining [referential integrity](#) and [other constraints](#).

Creating a Table

Refer to the *Staff* relation in the *DreamHome* case study provided in your reference document.

- The CREATE TABLE statement creates a table called *Staff* consisting of one or more columns of the specified *dataType*.

```
CREATE TABLE Staff ( staffNo CHAR(5),  
                        fName VARCHAR(20),  
                        lName VARCHAR(20),  
                        position VARCHAR(20),  
                        sex CHAR(1),  
                        DOB DATE,  
                        salary DECIMAL(10, 2),  
                        branchNo CHAR(4));
```

Integrity Enhancement Feature

- Integrity control **consists of constraints** that you wish to impose to protect the database from becoming inconsistent.
- **Types** of integrity constraint
 1. required data
 2. domain constraints
 3. entity integrity
 4. referential integrity
 5. general constraints
- These constraints can be defined in the CREATE and ALTER TABLE statements.

Integrity Enhancement Feature

1. Required data

- Some columns must contain a valid value (not allowed to contain nulls).

Example: every member of staff must have an associated job position (Manager, Assistant, and so on).

- Use **NOT NULL** column specifier in the CREATE and ALTER TABLE statements to provide this type of constraint.

Example: *position* **VARCHAR(10) NOT NULL**

2. Domain Constraints

- Every column has a domain (a set of legal values).

Example: Gender of a member of staff is either 'M' or 'F'.

- The ISO standard provides two mechanisms for specifying domains in the CREATE and ALTER TABLE statements.

Integrity Enhancement Feature

2. Domain Constraints (Cont..)

CHECK clause: Allows a constraint to be defined on a column or the entire table.

- The format of the CHECK clause is: **CHECK** (searchCondition)

Example: sex **CHAR NOT NULL CHECK** (sex **IN** ('M', 'F'))

CREATE DOMAIN statement:

```
CREATE DOMAIN DomainName [AS] dataType  
[DEFAULT defaultOption]  
[CHECK (searchCondition)]
```

- For the above example, you could define a domain for sex as

```
CREATE DOMAIN SexType AS CHAR
```

```
DEFAULT 'M'
```

```
CHECK (VALUE IN ('M', 'F'));
```

- When defining the column sex, use the domain name *SexType* in place of the data type CHAR:
Ex: sex *SexType* **NOT NULL**

Integrity Enhancement Feature

2. Domain Constraints (Cont..)

- Domains can be removed from the database.

```
DROP DOMAIN DomainName [RESTRICT | CASCADE]
```

- **RESTRICT** or **CASCADE**: specifies the action to be taken if the domain is currently being used.
- If **RESTRICT** is specified and the domain is used in an existing table, view, or assertion definition, the drop will fail.
- In **CASCADE**, the table column that is based on the domain is automatically changed to use the domain's underlying data type, and any constraint or default clause for the domain is replaced by a column constraint or column default clause.

Integrity Enhancement Feature

3. Entity integrity

- The primary key of a table must contain a **unique, non-null** value for each row.

Example: Define the primary key of the *PropertyForRent* table

PRIMARY KEY(propertyNo)

Define a composite primary key (clientNo and propertyNo) of the *Viewing* table:

PRIMARY KEY(clientNo, propertyNo)

- The PRIMARY KEY clause can be specified only once per table.
- It is still possible to ensure uniqueness for any alternate keys in the table using the keyword **UNIQUE**.

UNIQUE (clientNo, propertyNo)

- Every column that appears in a UNIQUE clause must also be declared as NOT NULL.

Integrity Enhancement Feature

4. Referential Integrity

- Referential integrity means that, if the foreign key contains a value, that value must refer to an existing, valid row in the parent table.

Example: Define the foreign key *branchNo* of the *PropertyForRent* table

FOREIGN KEY(branchNo) **REFERENCES** Branch

- **SQL rejects** any INSERT or UPDATE operation that attempts to **create a foreign key value in a child table** without a matching candidate key value in the parent table.
- Any UPDATE or DELETE operation that **attempts to update or delete a candidate key value in the parent** table that has some matching rows in the child table is dependent on the **referential action** specified using the **ON UPDATE** and **ON DELETE** subclauses of the FOREIGN KEY clause.

Integrity Enhancement Feature

4. Referential Integrity (Cont..)

Example: When the user attempts to **delete** a row from a parent table, and there are one or more matching rows in the child table, SQL supports **four options** regarding the action to be taken:

- **CASCADE**: Delete the row from the parent table and automatically delete the matching rows in the child table. Since these deleted rows may themselves have a candidate key that is used as a foreign key in another table, the foreign key rules for these tables are triggered, and so on, in a cascading manner.
- **SET NULL**: Delete the row from the parent table and set the foreign key value(s) in the child table to *NULL*. This is valid only if the foreign key columns do not have the *NOT NULL* qualifier specified.
- **SET DEFAULT**: Delete the row from the parent table and set each component of the foreign key in the child table to the specified default value. This is valid only if the foreign key columns have a *DEFAULT* value specified.
- **NO ACTION**: Reject the delete operation from the parent table. This is the default setting if the *ON DELETE* rule is omitted.

Integrity Enhancement Feature

4. Referential Integrity (Cont..)

Example: In the *PropertyForRent* table, the staff number *staffNo* is a foreign key referencing the *Staff* table. Specify a **deletion rule** such that, if a staff record is deleted from the *Staff* table, the values of the corresponding *staffNo* column in the *PropertyForRent* table are set to NULL:

FOREIGN KEY (*staffNo*) **REFERENCES** *Staff* **ON DELETE SET NULL**

Example: The owner number *ownerNo* in the *PropertyForRent* table is a foreign key referencing the *PrivateOwner* table. Specify an **update rule** such that, if an owner number is updated in the *PrivateOwner* table, the corresponding column(s) in the *PropertyForRent* table are set to the new value:

FOREIGN KEY (*ownerNo*) **REFERENCES** *PrivateOwner* **ON UPDATE CASCADE**

Integrity Enhancement Feature

5. General constraints

Example: *DreamHome* may have a rule that prevents a member of staff from managing more than 100 properties at the same time.

- The general constraints can be specified using the **CHECK** and **UNIQUE** clauses of the CREATE and ALTER TABLE statements and the **CREATE ASSERTION** statement.
- The CREATE ASSERTION statement is an integrity constraint that is not directly linked with a table definition.
- The format of the statement is:

```
CREATE ASSERTION AssertionName  
CHECK (searchCondition)
```

Example:

```
CREATE ASSERTION StaffNotHandlingTooMuch  
CHECK (NOT EXISTS (SELECT staffNo  
                     FROM PropertyForRent  
                     GROUP BY staffNo  
                     HAVING COUNT(*) > 100))
```

Creating a Table

Activity 2: Create the same *Staff* relation, defining **referential integrity** and **other constraints**.

```
CREATE TABLE Staff ( staffNo CHAR(5) NOT NULL,  
                      fName VARCHAR(20) NOT NULL,  
                      lName VARCHAR(20) NOT NULL,  
                      position VARCHAR(20),  
                      sex CHAR(1),  
                      DOB DATE,  
                      salary DECIMAL(10, 2),  
                      branchNo CHAR(4),
```

```
CONSTRAINT pk_staff PRIMARY KEY (staffNo),
```

```
CONSTRAINT fk_staff_branch FOREIGN KEY (branchNo) REFERENCES Branch(branchNo));
```

- The **table constraints** can optionally be preceded by the clause:

CONSTRAINT ConstraintName

which allows the constraint to be dropped by name using the ALTER TABLE statement.

Changing Table Definitions

- The ISO standard provides an **ALTER TABLE** statement for changing the structure of a table once it has been created.
- The definition of the ALTER TABLE statement in the ISO standard consists of six options:
 - add a new column to a table
 - drop a column from a table
 - add a new table constraint
 - drop a table constraint
 - set a default for a column
 - drop a default for a column

The basic syntax:

```
ALTER TABLE TableName  
[ADD [COLUMN] columnName dataType [NOT NULL] [UNIQUE]  
[DEFAULT defaultOption] [CHECK (searchCondition)]]  
[DROP [COLUMN] columnName [RESTRICT | CASCADE]]  
[ADD [CONSTRAINT [ConstraintName]] tableConstraintDefinition]  
[DROP CONSTRAINT ConstraintName [RESTRICT | CASCADE]]  
[ALTER [COLUMN] SET DEFAULT defaultOption]  
[ALTER [COLUMN] DROP DEFAULT]
```

Changing Table Definitions

Example 1: Add a new column to a table

Change the *Client* table by adding a new column, *prefNoRooms*, representing the preferred number of rooms.

```
ALTER TABLE Client ADD prefNoRooms SMALLINT;
```

Example 2: Set a default for a column

Change the *Staff* table by setting the default for the *sex* column to female ('F').

```
ALTER TABLE Staff ALTER sex SET DEFAULT 'F';
```

Example 3: Drop a default for a column

Change the *Staff* table by removing the default of 'Assistant' for the *position* column.

```
ALTER TABLE Staff ALTER position DROP DEFAULT;
```

Changing Table Definitions

Example 4: Add a new table constraint

- A *tableConstraintDefinition* is one of the clauses: PRIMARY KEY, UNIQUE, FOREIGN KEY, or CHECK.

```
ALTER TABLE TableName  
[ADD [CONSTRAINT [ConstraintName]] tableConstraintDefinition]
```

```
ALTER TABLE PropertyForRent ADD CONSTRAINT chk_rooms CHECK (rooms >= 1);
```

Example 5: Drop a table constraint

Change the *PropertyForRent* table by removing the constraint *StaffNotHandlingTooMuch* that prevents staff from handling more than 100 properties at a time.

```
ALTER TABLE PropertyForRent DROP CONSTRAINT StaffNotHandlingTooMuch;
```

Changing Table Definitions

Example 6: Drop a column from a table

- The DROP COLUMN has an optional qualifier to specify the DROP action.
 - **RESTRICT**: The DROP operation is rejected if the column is referenced by another database object (e.g., by a view definition). This is the default setting.
 - **CASCADE**: The DROP operation proceeds and automatically drops the column from any database objects it is referenced by. This operation cascades, so that if a column is dropped from a referencing object, SQL checks whether *that* column is referenced by any other object and drops it from there if it is, and so on.

Example: Change the *Client* table by removing the column, *prefNoRooms*.

ALTER TABLE Client DROP prefNoRooms;

ALTER TABLE Client DROP prefNoRooms CASCADE;

Removing a Table

- Remove a table from the database using the DROP TABLE statement.

```
DROP TABLE TableName [RESTRICT | CASCADE]
```

Example: Remove the table *PropertyForRent* from the database

```
DROP TABLE PropertyForRent;
```

This command removes not only the named table, but also all the rows within it.

- The DROP TABLE statement allows you to specify whether the DROP action applies.
 - **RESTRICT**: The DROP operation is rejected if there are any other objects that depend for their existence upon the continued existence of the table to be dropped.
 - **CASCADE**: The DROP operation proceeds, and automatically drops all dependent objects (and objects dependent on these objects).
- One common use of DROP TABLE is to correct mistakes made when creating a table.

2. Data Manipulation Language (DML)

Data Manipulation Language (DML)

- Performs database maintenance.
- These SQL statements are used to add, change, select and remove data from the database (manipulate existing data objects).

Example:

SELECT	to query data in the database
INSERT	to insert data into a table
UPDATE	to update data in a table
DELETE	to delete data from a table

- INSERT, UPDATE, DELETE SQL statements modify the contents of the tables in the database.

Literals

- Literals are constants that are used in SQL statements. There are different literal forms for every data type supported by SQL (see Section 6.1.1).
- However, for simplicity, we can distinguish between literals that are enclosed in single quotes and those that are not.
 - All non-numeric data values must be enclosed in single quotes
 - All numeric data values must **not** be enclosed in single quotes.

Example: Use literals to insert data into a table.

```
INSERT INTO PropertyForRent(propertyNo, street, city, postcode, type, rooms, rent,  
ownerNo, staffNo, branchNo)  
VALUES ('PA14', '16 Holhead', 'Aberdeen', 'AB7 5SU', 'House', 6, 650.00, 'CO46', 'SA9',  
'B007');
```

Adding data to the database

- The **INSERT** statement allows to insert a single or multiple rows into a named table with the following format:

```
INSERT INTO TableName [(columnList)]  
VALUES (dataValueList)
```

- **TableName** may be either a base table or an updatable view.
- **columnList** represents a list of one or more column names separated by commas [Optional]
 - If omitted, SQL assumes a list of all columns in their original CREATE TABLE order.
 - If specified, then any columns that are omitted from the list must have been declared as **NULL** columns when the table was created, unless the DEFAULT option was used when creating the column.
- **dataValueList** must match the *columnList* as follows:
 - The number of items in each list must be the same.
 - There must be a direct correspondence in the position of items in the two lists.
 - The data type of each item in *dataValueList* must be compatible with the data type of the corresponding column.

Adding data to the database

Example 1: Insert a new row into the *Staff* table, supplying data for all columns.

INSERT INTO Staff

VALUES ('SG16', 'Alan', 'Brown', 'Assistant', 'M', '1957-05-25', 8300, 'B003'); **or**

INSERT INTO Staff (StaffNo, fName, lName, position, sex, DOB, salary, branchNo)

VALUES ('SG16', 'Alan', 'Brown', 'Assistant', 'M', '1957-05-25', 8300, 'B003');

Example 2: Insert a new row into the *Staff* table, supplying data for all mandatory columns: *staffNo, fName, lName, position, salary, and branchNo*.

INSERT INTO Staff (staffNo, fName, lName, position, salary, branchNo)

VALUES ('SG44', 'Anne', 'Jones', 'Assistant', 8100, 'B003'); **or**

INSERT INTO Staff

VALUES ('SG44', 'Anne', 'Jones', 'Assistant', **NULL**, **NULL**, 8100, 'B003');

* This query explicitly specified that the columns *sex* and *DOB* should be set to **NULL**.

Adding data to the database

Example 3: Insert more than one row into the *Staff* table, supplying data for all columns.

INSERT INTO Staff

```
VALUES ('SG16', 'Alan', 'Brown', 'Assistant', 'M', '1957-05-25', 8300, 'B003'),  
        ('SL21', 'John', 'White', 'Manager', 'M', '1945-10-01', 30000, 'B005'),  
        ('SG37', 'Ann', 'Beech', 'Assistant', 'F', '1960-11-10', 12000, 'B003'),  
        ('SG14', 'David', 'Ford', 'Supervisor', 'M', '1958-03-24', 18000, 'B003');
```

Activity 3: Insert the newly opened branch details into the *Branch* table.

Branch number: B008, Branch Address: 81 RedDeer St., Calgary, CA89 6EZ.

```
INSERT INTO Branch VALUES ('B008 ', '81 RedDeer St.', 'Calgary', ' CA89 6EZ ');
```

Modifying data in the database

- The **UPDATE** statement allows the contents of existing rows in a named table to be changed.

```
UPDATE TableName  
SET columnName1 = dataValue1 [, columnName2 = dataValue2 . . . ]  
[WHERE searchCondition]
```

- **TableName** can be the name of a base table or an updatable view.
- The **SET** clause specifies the names of one or more columns that are to be updated.
- The **WHERE** clause is optional.
 - If omitted, the named columns are updated for *all* rows in the table.
 - If specified, only those rows that satisfy the *searchCondition* are updated.
 - The new *dataValue(s)* must be compatible with the data type(s) for the corresponding column(s).

Modifying data in the database

Example 1: UPDATE all rows

Give all staff a 3% pay increase to all members in the staff.

UPDATE Staff

SET salary = salary*1.03;

Example 2: UPDATE specific rows

Give all Managers a 5% pay increase.

UPDATE Staff

SET salary = salary*1.05

WHERE position = 'Manager';

The WHERE clause selects rows that contain data for Managers, and the update salary = salary*1.05 is applied only to those rows.

Modifying data in the database

Example 3: UPDATE multiple columns

Promote *David Ford* (*staffNo* = 'SG14') to *Manager* and change his salary to £20,000.

UPDATE Staff

SET position = 'Manager', salary = 20000

WHERE staffNo = 'SG14';

Activity 4: Update the rent for property PG36 to 400.

UPDATE PropertyForRent

SET rent = 400

WHERE propertyNo = 'PG36';

Deleting data from the database

- The **DELETE** statement allows rows to be deleted from a named table.

```
DELETE FROM TableName  
[WHERE searchCondition]
```

- The *searchCondition* is optional.
 - If omitted, all rows are deleted from the table (does not delete the table itself – to delete the table contents and the table definition, the DROP TABLE statement must be used instead).
 - If a *searchCondition* is specified, only those rows that satisfy the condition are deleted.

Deleting data from the database

Example 1: DELETE specific rows

Delete all *viewings* that relate to property PG4.

DELETE FROM Viewing

WHERE propertyNo = 'PG4';

- The WHERE clause finds the rows for property PG4, and the delete operation is applied only to these particular rows.

Example 2: DELETE all rows

Delete all rows from the *Viewing* table.

DELETE FROM Viewing;

- No WHERE clause has been specified. So the delete operation applies to all rows in the table.
- This leaves only the table definition, so that you are able to insert data into the table at a later stage.

Deleting data from the database

Activity 4: Delete all female members from the staff.

```
DELETE FROM Staff WHERE sex = 'F';
```

Activity 5: Delete all viewings without any comments.

```
DELETE FROM Viewing WHERE comment IS NULL;
```

Selecting data

- The purpose of the **SELECT** statement is to retrieve and display data from one or more database tables.
- SELECT is the most frequently used SQL command and has the following general form:

```
SELECT      [DISTINCT | ALL] { * | [columnExpression [AS newName]] [, ... ] }  
FROM      TableName [alias] [, ... ]  
[WHERE      condition]  
[GROUP BY  columnList] [HAVING condition]  
[ORDER BY  columnList]
```

- *columnExpression* represents a column name or an expression
- *alias* is an optional abbreviation for *TableName*

Selecting data

- The **sequence of processing** in a **SELECT** statement is:
 - FROM specifies the table or tables to be used
 - WHERE filters the rows subject to some condition
 - GROUP BY forms groups of rows with the same column value
 - HAVING filters the groups subject to some condition
 - SELECT specifies which columns are to appear in the output
 - ORDER BY specifies the order of the output

Mandatory	{	SELECT	[DISTINCT ALL]	{ * [columnExpression [AS newName]] [, . . .] }
	}	FROM	TableName [alias] [, . . .]	
Optional	{	[WHERE	condition]	
		[GROUP BY	columnList] [HAVING condition]	
		[ORDER BY	columnList]	

Selecting data

Example 1: Retrieve all rows, all columns

List full details of all staff.

- Since there are no restrictions specified in this query, the WHERE clause is unnecessary, and all columns are required.

```
SELECT staffNo, fName, lName, position, sex, DOB, salary, branchNo
```

```
FROM Staff;
```

- There is a quick way of expressing 'all columns' in SQL, using an asterisk (*) in place of the column names.

```
SELECT *
```

```
FROM Staff;
```

Result:

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000.00	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000.00	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000.00	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000.00	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000.00	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000.00	B005

Selecting data

Example 2: Retrieve specific columns, all rows

Produce a list of salaries for all staff, showing only the staff number, the first and last names, and the salary details.

```
SELECT staffNo, fName, lName, salary  
FROM Staff;
```

- A new table is created from Staff containing only the designated columns *staffNo*, *fName*, *lName*, and *salary*, in the specified order.
- Unless specified, the rows in the result table may not be sorted.

Result:

staffNo	fName	lName	salary
SL21	John	White	30000.00
SG37	Ann	Beech	12000.00
SG14	David	Ford	18000.00
SA9	Mary	Howe	9000.00
SG5	Susan	Brand	24000.00
SL41	Julie	Lee	9000.00

Selecting data

Example 3: Use of DISTINCT

List the property numbers of all properties that have been viewed.

```
SELECT propertyNo
```

```
FROM Viewing;
```

- Notice that there are several duplicates because **SELECT** does not eliminate duplicates when it projects over one or more columns.
- To eliminate the duplicates, use the **DISTINCT** keyword.

```
SELECT DISTINCT propertyNo
```

```
FROM Viewing;
```

propertyNo
PA14
PG4
PG4
PA14
PG36

Result

propertyNo
PA14
PG4
PG36

Result with
DISTINCT

Selecting data

Example 4: Calculated fields (computed or derived field)

Produce a list of monthly salaries for all staff, showing staff number, first and last names, and salary details.

```
SELECT staffNo, fName, lName, salary/12
```

```
FROM Staff;
```

- A calculated field specifying an SQL expression is generally included in the SELECT list.
- An SQL expression can involve +, -, *, /, and () can be used to build complex expressions.
- More than one table column can be used in a calculated column; they must have a numeric type.

Result:

staffNo	fName	lName	col4
SL21	John	White	2500.00
SG37	Ann	Beech	1000.00
SG14	David	Ford	1500.00
SA9	Mary	Howe	750.00
SG5	Susan	Brand	2000.00
SL41	Julie	Lee	750.00

Selecting data

Example 5: Calculated fields (computed or derived field) (Cont...)

- Some dialects give the column a name corresponding to its position in the table, or leave the column name blank or use the expression entered in the SELECT list.
- The ISO standard allows the column to be named using an AS clause.

```
SELECT staffNo, fName, lName, salary/12 AS monthlySalary  
FROM Staff;
```

- In this case, the column heading of the result table would be *monthlySalary* rather than col4.

Result:

staffNo	fName	lName	Monthly Salary
SL21	John	White	2500.00
SG37	Ann	Beech	1000.00
SG14	David	Ford	1500.00
SA9	Mary	Howe	750.00
SG5	Susan	Brand	2000.00
SL41	Julie	Lee	750.00

Selecting data

Activity 6: List the unique types of properties available for renting.

```
SELECT DISTINCT type FROM PropertyForRent;
```

Activity 7: All staff members are required to pay 3% tax on their salaries. Write an SQL query to display the following details for all staff members:

- Gross Salary, displayed as *grossSalary*
- Tax Deduction Amount (3%), displayed as *taxPayment*
- Net Payable Salary after Tax, displayed as *payable*. The payable salary should be calculated by subtracting the tax payment from the gross salary.

```
SELECT staffNo, salary AS grossSalary, (salary * 0.03) AS taxPayment, (salary - (salary *  
0.03)) AS payable  
FROM Staff;
```

Selecting data – WHERE clause

- **WHERE** clause restricts the rows that are retrieved using a search condition that specifies the rows to be retrieved.
- The five basic search conditions (or *predicates* using the ISO terminology) are as follows:
 - *Comparison*: Compare the value of one expression to the value of another expression.
 - *Range*: Test whether the value of an expression falls within a specified range of values (BETWEEN/NOT BETWEEN).
 - *Set membership*: Test whether the value of an expression equals one of a set of values (IN/NOT IN).
 - *Pattern match*: Test whether a string matches a specified pattern (LIKE/NOT LIKE).
 - *Null*: Test whether a column has a null (unknown) value (IS NULL/IS NOT NULL).

Selecting data – WHERE clause

- Simple **comparison operators** are available in SQL:
 - = equals
 - < > is not equal to (ISO standard)
 - ! = is not equal to (allowed in some dialects)
 - < is less than
 - < = is less than or equal to
 - > is greater than
 - > = is greater than or equal to
- More complex predicates can be generated using the logical operators **AND**, **OR**, and **NOT**, with parentheses (if needed or desired) to show the order of evaluation.
- The rules for evaluating a conditional expression are:
 - an expression is evaluated left to right
 - subexpressions in brackets are evaluated first
 - NOTs are evaluated before ANDs and ORs
 - ANDs are evaluated before ORs.
- The use of parentheses is always recommended to remove any possible ambiguities.

Selecting data – WHERE clause

Example 6: Comparison search condition

List all staff with a salary greater than 10,000.

```
SELECT staffNo, fName, lName,  
position, salary
```

```
FROM Staff
```

```
WHERE salary > 10000;
```

Result:

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG37	Ann	Beech	Assistant	12000.00
SG14	David	Ford	Supervisor	18000.00
SG5	Susan	Brand	Manager	24000.00

Example 7: Compound comparison search condition

List the addresses of all branch offices in London or Glasgow.

```
SELECT *
```

```
FROM Branch
```

```
WHERE city = 'London' OR city =  
'Glasgow';
```

Result:

branchNo	street	city	postcode
B005	22 Deer Rd	London	SW1 4EH
B003	163 Main St	Glasgow	G11 9QX
B002	56 Clover Dr	London	NW10 6EU

Selecting data – WHERE clause

Activity 8: List all male employees in the staff with a salary greater than or equal to 30,000.

```
SELECT *  
FROM Staff  
WHERE sex = 'M' AND salary >= 30000;
```

Selecting data – WHERE clause

Example 8: Range search condition (BETWEEN/NOT BETWEEN)

List all staff with a salary between 20,000 and 30,000.

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary BETWEEN 20000 AND 30000;
```

- You could have expressed the above query as:

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary >= 20000 AND salary <= 30000;
```

- However, the BETWEEN test is a simpler way to express a search condition when considering a range of values.

Result:

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG5	Susan	Brand	Manager	24000.00

Selecting data – WHERE clause

Example 9: Set membership search condition (IN/NOT IN)

List all managers and supervisors.

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE position IN ('Manager', 'Supervisor');
```

Result:

staffNo	fName	lName	position
SL21	John	White	Manager
SG14	David	Ford	Supervisor
SG5	Susan	Brand	Manager

- **IN** tests whether a data value matches one of a list of values, in this case either 'Manager' or 'Supervisor'.
- There is a negated version (**NOT IN**) that can be used to check for data values that do not lie in a specific list of values.
- Like **BETWEEN**, the **IN** test does not add much to the expressive power of SQL.

Selecting data – WHERE clause

Example 9: Set membership search condition (IN/NOT IN) (Cont...)

- You could have expressed the above query as:

SELECT staffNo, fName, lName, position

FROM Staff

WHERE position = 'Manager' **OR** position = 'Supervisor';

- However, the **IN** test provides a more efficient way of expressing the search condition, particularly if the set contains many values.

Selecting data – WHERE clause

Example 10: Pattern match search condition (LIKE/NOT LIKE)

Find all owners with the string 'Glasgow' in their address.

- For this query, you must search for the string 'Glasgow' appearing somewhere within the *address* column of the *PrivateOwner* table.
- SQL has two special pattern-matching symbols:
 - % character represents any sequence of zero or more characters (*wildcard*).
 - _ underscore character represents any single character.

```
SELECT ownerNo, fName, lName, address, telNo
```

```
FROM PrivateOwner
```

```
WHERE address LIKE '%Glasgow%';
```

Selecting data – WHERE clause

Example 10: Pattern match search condition (LIKE/NOT LIKE) (Cont...)

- All other characters in the pattern represent themselves.

For example:

- address LIKE 'H%' means the first character must be *H*, but the rest of the string can be anything.
- address LIKE 'H____' means that there must be exactly four characters in the string, the first of which must be an *H*.
- address LIKE '%e' means any sequence of characters, of length at least 1, with the last character an *e*.
- address LIKE '%Glasgow%' means a sequence of characters of any length containing *Glasgow*.
- address NOT LIKE 'H%' means the first character cannot be an *H*.

Selecting data – WHERE clause

Example 11: NULL search condition (IS NULL/IS NOT NULL)

List the details of all viewings on property PG4 where a comment has not been supplied.

- A **null** comment is considered to have an unknown value, so you cannot test whether it is equal or not equal to another string.
- Test for **null** explicitly using the special keyword **IS NULL**:

```
SELECT clientNo, viewDate
```

```
FROM Viewing
```

```
WHERE propertyNo = 'PG4' AND comment IS NULL;
```

- The negated version (IS NOT NULL) can be used to test for values that are not null.

Result:

clientNo	viewDate
CR56	26-May-04

Sorting Results

- The rows of an SQL query result table are not arranged in any particular order (some DBMSs may use a default ordering based on a primary key).
- The results of a query can be sorted using the **ORDER BY** clause in the SELECT statement.
- The **ORDER BY** clause consists of a list of **column identifiers** that the result is to be sorted on, separated by commas.
 - A column identifier may be either a column name or a column number
 - The retrieved rows ordered in ascending (**ASC**) or descending (**DESC**) order on any column or combination of columns, regardless of whether that column appears in the result.
- The ORDER BY clause must always be the last clause of the SELECT statement.

Sorting data

Example 12: Single-column ordering

Produce a list of salaries for all staff, arranged in descending order of salary.

```
SELECT staffNo, fName, lName, salary
FROM Staff
ORDER BY salary DESC;
```

- The output is to be arranged in descending order of salary.
- You could have expressed the ORDER BY clause as: **ORDER BY 4 DESC**, with the 4 relating to the fourth column name in the SELECT list, namely salary.
- Column numbers are a deprecated feature of the ISO standard and should not be used.

Result:

staffNo	fName	lName	salary
SL21	John	White	30000.00
SG5	Susan	Brand	24000.00
SG14	David	Ford	18000.00
SG37	Ann	Beech	12000.00
SA9	Mary	Howe	9000.00
SL41	Julie	Lee	9000.00

Sorting data

Example 13: Multiple column ordering

- It is possible to include more than one element in the ORDER BY clause.
 - The **major sort key** determines the overall order of the result table.
 - The second element, which appears in the ORDER BY clause, is called a **minor sort key**.

Produce an abbreviated list of properties arranged in order of property type.

PropertyForRent

propertyNo	street	city	postcode	type	rooms	rent	ownerNo
PA14	16 Holhead	Aberdeen	AB7 5SU	House	6	650	CO46
PL94	6 Argyll St	London	NW2	Flat	4	400	CO87
PG4	6 Lawrence St	Glasgow	G11 9QX	Flat	3	350	CO40
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375	CO93
PG21	18 Dale Rd	Glasgow	G12	House	5	600	CO87
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450	CO93

SELECT propertyNo, type, rooms, rent
FROM PropertyForRent
ORDER BY type;

Result:

propertyNo	type	rooms	rent
PL94	Flat	4	400
PG4	Flat	3	350
PG36	Flat	3	375
PG16	Flat	4	450
PA14	House	6	650
PG21	House	5	600

Sorting data

Example 13: Multiple column ordering (Cont...)

- There are four flats in this list. As we did not specify any minor sort key, the system arranges these rows in any order it chooses.
- Specify a minor order to arrange the properties in order of rent, as follows:

```
SELECT propertyNo, type, rooms, rent
```

```
FROM PropertyForRent
```

```
ORDER BY type, rent DESC;
```

- Now, the result is ordered first by property type, in ascending alphabetic order (ASC being the default setting), and within property type, in descending order of rent.

Result:

propertyNo	type	rooms	rent
PG16	Flat	4	450
PL94	Flat	4	400
PG36	Flat	3	375
PG4	Flat	3	350
PA14	House	6	650
PG21	House	5	600

Sorting data

Activity 9: List only the *Flats* for rent, arranged in order of number of rooms and rent in descending order.

```
SELECT *  
FROM PropertyForRent  
WHERE type = 'Flat'  
ORDER BY rooms DESC, rent DESC;
```

Any questions?